Software
Systems
Engineering

Universität Hildesheim

Web-Technologies

Chapter 2 – Application Stacks and Environments

Holger Eichelberger, Klaus Schmid

{eichelberger, schmid}@sse.uni-hildesheim.de

Overview

Typically, Web-Applications utilize several technologies at the same time. Other technologies are used for supporting the development.

- Typical Web-Stacks
- The Web-Stack for this course
- Development Environments

Goal: Setup of the Web-Stack and the own development environment.

© AG SSE

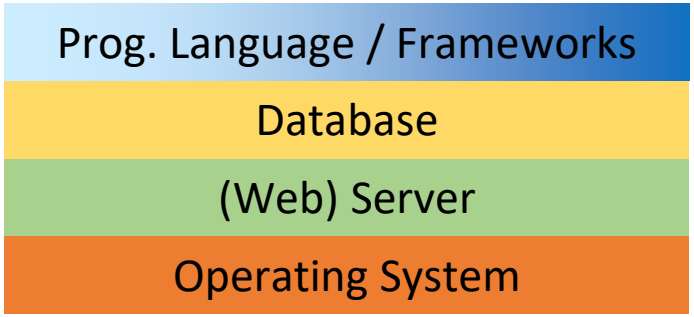
Contents

1. Introduction 2
2. Web Stacks 3
3. IDEs 9
4. Virtual Machines 15
5. A bit Node.js 21
6. Summary 28

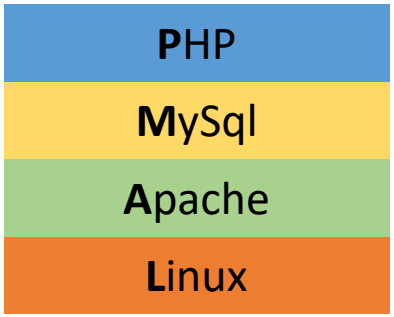
© AG SSE

Web Stack

- A **Web stack** is the collection of software required for **Web** development. At least, a **Web stack** contains an operating system (OS), a programming language, database software and a **Web** server.



- A rather common, traditional Stack is **LAMP**



https://whatis.techtarget.com/definition/Web-stack

© AG SSE

Some Further Traditional Web Stacks

JavaEE/JSP	ASP.NET	PHP/Ruby*	Django/Python
MySQL/PostgreSQL	SQL Server	MySQL	MySQL
Apache/Tomcat	IIS	Apache	Apache
Linux	Windows Server	Linux	Linux
Java	WISA	Ruby (*: Ruby, Ruby on Rails, RVM, JRuby)	Django (oo-Python)

- Strong dependency on operating system
- Native technologies
- Traditional SQL databases
- Often: Traditional languages, e.g., Java or .NET

© AG SSE

Some Modern Web Stacks

MEAN

AngularJS / Node.js	Node.js	Node.js	Bootstrap / Node.js
Express.js	Express.js	Express.js/JWT	Express.js/JWT/Redis
MongoDB	Google Datastore	EC2/PostgreSQL	MySQL, MongoDB
			Ngix
OS (usually Linux)	Google Cloud Complete	EC2	OS (Linux)

- Less focus on operating system, if not OS independent
- Lightweight (Open Source) technologies, e.g., Web frameworks, NoSQL, in-memory DBs
- Client-side technologies, e.g., JavaScript
- Cloud technologies, external services
- App- and hybrid technologies

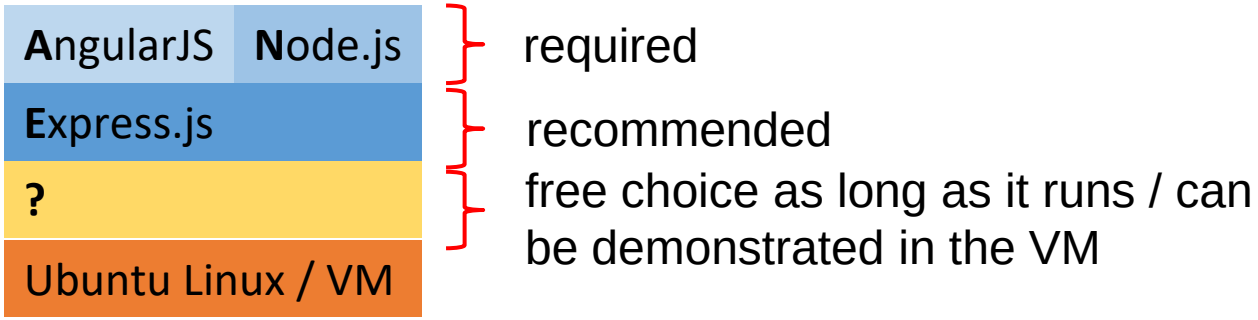
© AG SSE

Some Modern (Open Source) Technologies

- **AngularJS:** JavaScript/TypeScript-based front-end web application platform.
- **Bootstrap:** Web application front-end framework based on HTML / CSS templates for typography / navigation components.
- **Node.js:** Cross-platform JavaScript run-time environment that executes *JavaScript outside of a browser*.
- **Express.js:** Web application framework for Node.js, designed for building web applications and APIs.
- **JWT:** JSON Web Token is a JSON-based open standard for creating access tokens that assert some number of claims.
- **Redis:** In-memory data structure supporting abstract data structures, such as lists, maps, sets, sorted sets, hyperloglogs, bitmaps and spatial indexes

© AG SSE

Our Web “Stack”: NAE*U



© AG SSE

Contents

1. Introduction 2
2. Web Stacks 3
3. IDEs 9
4. Virtual Machines 15
5. A bit Node.js 21
6. Summary 28

© AG SSE

IDEs

Integrated Develoment Environments (IDE)

- IDE: Software application that provides comprehensive facilities for software development, typically including
  - Source code editor(s)
  - Build automation tools
  - Debugger
  - (Intelligent) Code completion
  - Browsing facilities
- In contrast to beginner courses, we are more flexible regarding IDE and server environment.
- Choice of the IDE in this course is free
- We make some (rational) suggestions

© AG SSE

IDEs

(Our) Comparison Requirements

- Support for modern frameworks (Angular, Node.js)
- Support for underlying languages (HTML, JavaScript, TypeScript, CSS)
- Code auto completion, navigation, refactoring, debugging
- Integration support for version management
- Integration of command line / terminal
- Availability (if possible, Open Source)
- Platform-Independence (at least Windows, Linux, MacOS)
- Support for Extensions / Plugins
- Editor Performance

© AG SSE

IDEs

(Our) Candidates

- Visual Studio Code (VSCode) by Microsoft (open source)
- Atom by GitHub (open source)
- Eclipse by Eclipse Foundation (open source), with plugins
- WebStorm (WS) by JetBrains (commercial, free license for students)
- Sublime Text (ST) by Sublime HQ (commercial)

© AG SSE

IDEs

(Our) Comparison Results

VSCode Completion IntelliSense seems to be more “intelligent”…

IDE	Frameworks	Languages	Completion	Navigation	Refactoring	Debugging	Version mgt.	Terminal	Availability	Platforms	Extensions	Performance*
VSCode	+	+	+	+	+	+	Git, SVN	++	OS	WLM	+	++
Atom	+	+	+	+	+	+	Git + GitHub	+	OS	WLM	+	+
Eclipse	+	+	+	+	+	+	Git, SVN, *	+	OS	WLM	+	--

++ integrated / very good, + plugin / ok, -- problematic

*e.g., https://blog.xinhong.me/post/sublime-text-vs-vscode-vs-atom-performance-dec-2016/

© AG SSE

IDEs

Recommendation

- VSCode or Atom
- Final selection is up to you, also regarding some decisions in the Web Stack.
- Installation… on VMs (next section)

© AG SSE

Contents

1. Introduction 2
2. Web Stacks 3
3. IDEs 9
4. Virtual Machines 15
5. A bit Node.js 21
6. Summary 28

© AG SSE

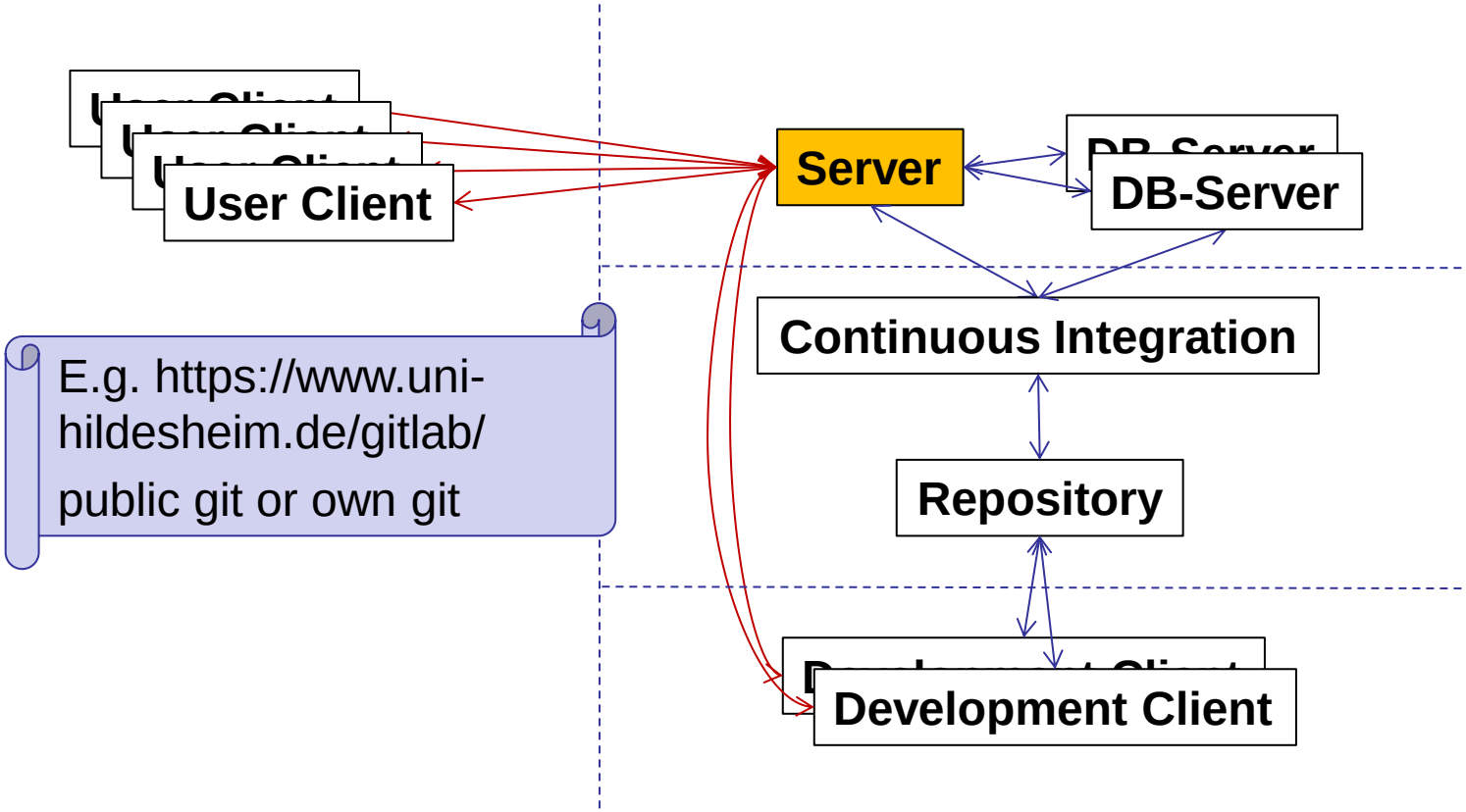
Why not native?

- We need to install some software: `npm`, IDE
- Demonstration/Grading installation:
  - Server software decision is „free“, e.g., express.js
  - Browser + Angular
  - Some may like containers, e.g., Docker (OS dependent, WSL)
- Allows for backups, can be executed as „server“
- For grading / artifacts:
  - Applying an incomplete setup/startup guideline is a no-go
  - Better, „full“ installation → VM

Experience: **No** Windows Guest VMs

© AG SSE

Environments



Virtual Machines

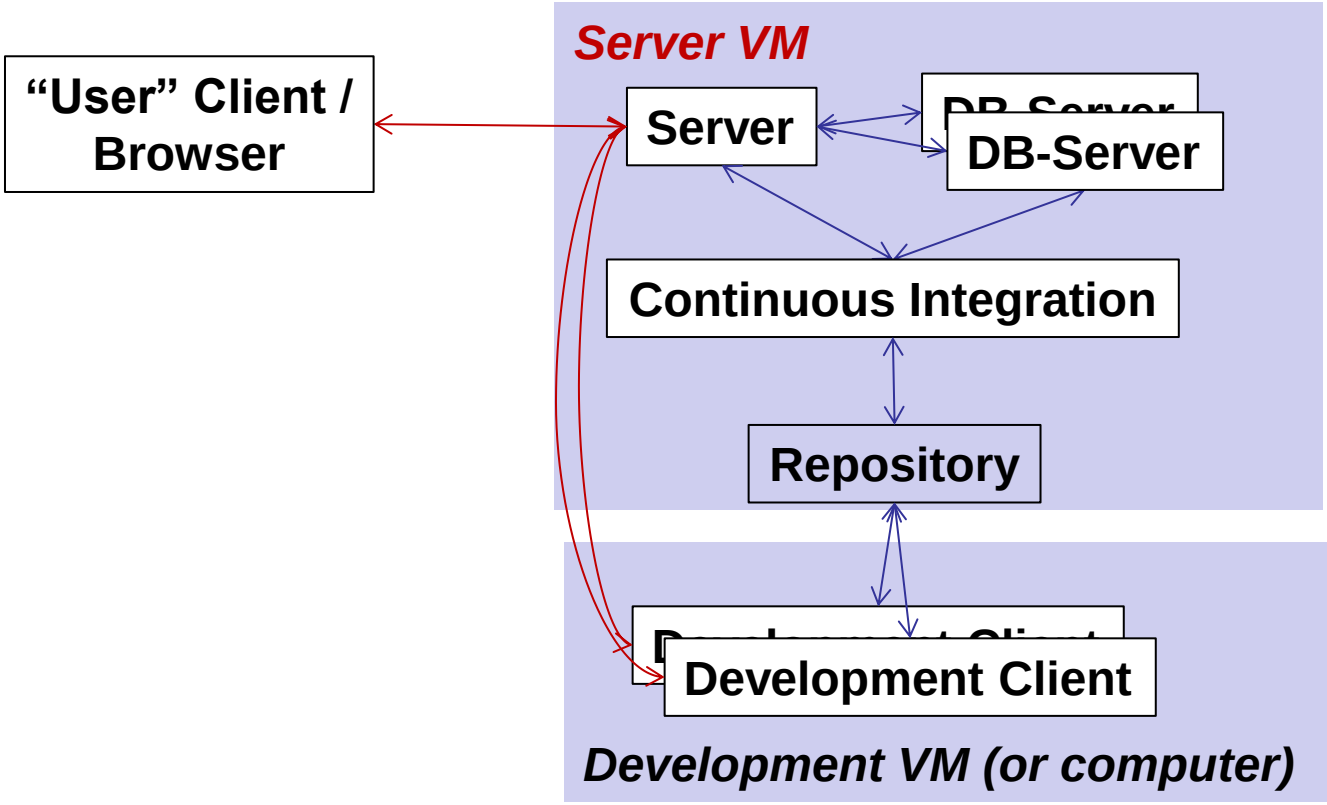
Do you know VMs? (see SE tools lab course)

- A virtual machine (VM) is a virtualized computer running in your operating system (**host**)
- Requires virtualization software, e.g., Virtual Box, VMWare Player
- An operating system setup running as **guest**
  - The harddrive of the guest is one or more files on your system
  - Memory, CPU and Network are shared (somehow)
  - Some interactions like drag & drop may work (specific tools)
- Thus, either
  - You install the guest operating system into the virtual harddrive
  - You download an already installed virtual machine (may require some setup work)

© AG SSE

Virtual Machines

One potential Installation for this Course



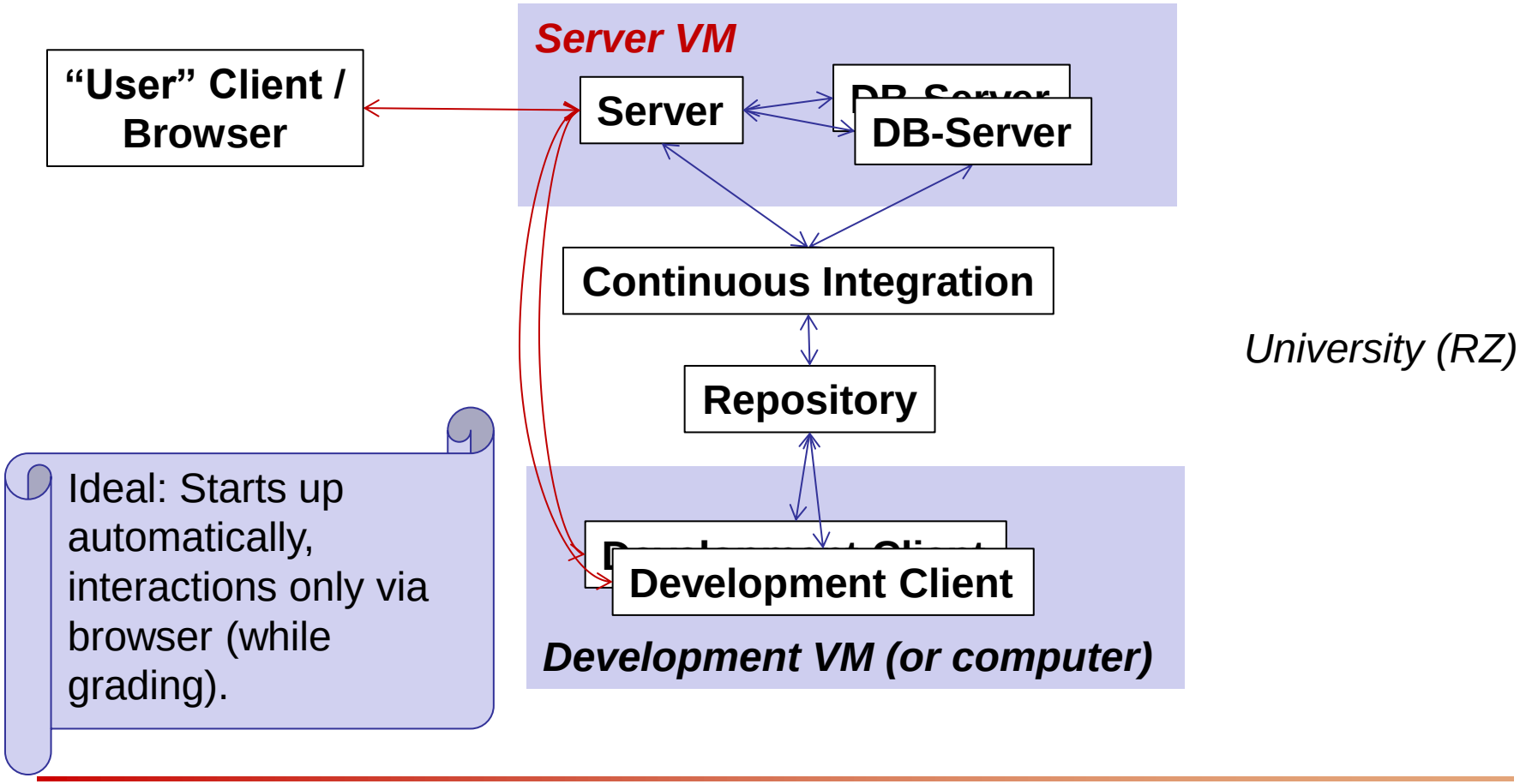
The VM plays the role of a **dedicated server**.

Simplified setup: Client may also be within VM.

© AG SSE

Virtual Machines

One potential Installation for this Course



© AG SSE

Installation

Disclaimer

- Following installation hints worked in the last editions of this course
- Shall do as basic installation
- Operating systems and software do change, e.g., package manager may do most of the work for you
- You may be happy with the versions provided by the operating system
- Important requirements for now:
  - Running node.js, npm and angular in some version
  - Gain basic VM experience and decide about versions for project later
  - Thus, no pre-installed VM from us, but basic VM available

© AG SSE

Outlook: Project Submission (1)

More details will follow.

- Technical side
  - Install recent Virtual Box >= 7.2.2
  - Recent Ubuntu Linux >= 25.04 VM from osboxes.org or https://praktikum.sse.uni-hildesheim.de/public/teaching/webtech/
  - Create a VM and replace the harddisk by the Ubuntu one (https://www.osboxes.org/guide/)
  - Recommendation: bridged network
  - Boot the VM, install guest additions
  - Install the required software
  - Keep all core parts local (Cloud is not always your friend)
  - Disable (unattended) updates
- Submission (academic cloud, fallback: harddrive, USB stick); details will follow

© AG SSE

Fallback

- Computers in SC.C.1.47 have VirtualBox installed
- Don’t store VM’s on Desktop/default folder (profile size limitation)
- Use `C:\Vmstud` (local machine only) and do a USB/cloud backup for transfer
- More network-based folders may follow

© AG SSE

Compatibility

But not newer, e.g., 20.19.0!

Angular	Node.js	Typescript	RXJS
20.2.x || 20.3.x	^20.19.0 || ^22.12.0 || ^24.0.0	>=5.9.0 <6.0.0	^6.5.3 || ^7.4.0
20.0.x || 20.1.x	^20.19.0 || ^22.12.0 || ^24.0.0	>=5.8.0 <5.9.0	^6.5.3 || ^7.4.0
19.2.x	^18.19.1 || ^20.11.1 || ^22.0.0	>=5.5.0 <5.9.0	^6.5.3 || ^7.4.0
19.1.x	^18.19.1 || ^20.11.1 || ^22.0.0	>=5.5.0 <5.8.0	^6.5.3 || ^7.4.0
19.0.x	^18.19.1 || ^20.11.1 || ^22.0.0	>=5.5.0 <5.7.0	^6.5.3 || ^7.4.0
18.1.x || 18.2.x	^18.19.1 || ^20.11.1 || ^22.0.0	>=5.4.0 < 5.6.0	^6.5.3 || ^7.4.0
18.0.x	^18.19.1 || ^20.11.1 || ^22.0.0	>=5.4.0 < 5.5.0	^6.5.3 || ^7.4.0

https://angular.io/guide/versions

© AG SSE

Installation (node.js, angular as Windows portable)

- Download node.js https://nodejs.org
- Unpack node.js
- Set PATH to extracted folder
- `node --version`
- `npm --version`
- `npm install -g @angular/cli@`*`version`*
- For powershell: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`
- *`ng version`*
- Similar for Linux, Mac

```
Microsoft Windows [Version 10.0.19045.3570]
(c) Microsoft Corporation. All rights reserved.

C:\Users\eichelberger>node --version
v18.10.0

C:\Users\eichelberger>npm --version
8.19.2

C:\Users\eichelberger>npm install -g @angular/cli

added 245 packages, and audited 246 packages in 7s

38 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\eichelberger>ng version

Angular CLI: 16.2.8
Node: 18.10.0
Package Manager: npm 8.19.2
OS: win32 x64

Angular:
...
```


© AG SSE

Installation (Visual Studio code as Windows portable)

- Visual Studio Code: https://code.visualstudio.com/docs/editor/portable
  - Download from https://code.visualstudio.com/download#
  - Unpack into a folder
  - Create a `data` folder within VS Code's folder
- Similar for Linux, Mac

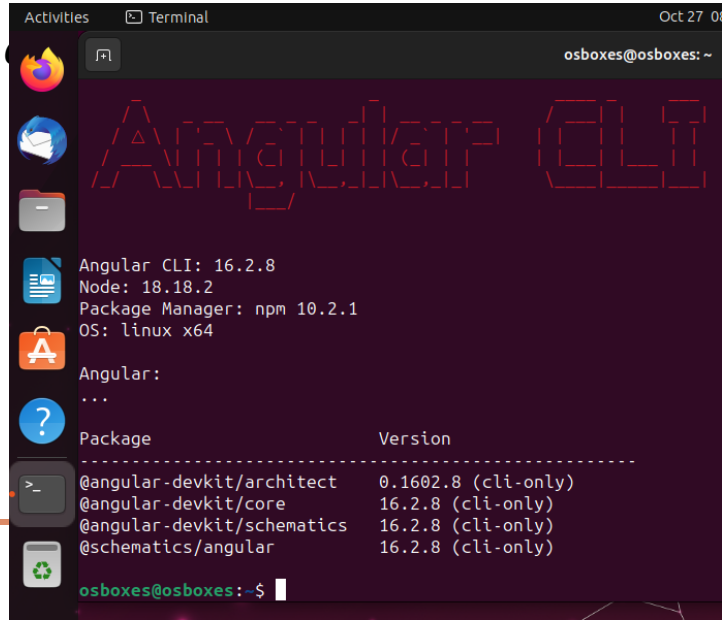
© AG SSE

Substitute *version appropriately.*

Installation Linux via package manager (2)

- `sudo apt-get update`
- `sudo apt-get install nodejs=`*`version`* `-y`
- *`node --version`*
- `sudo npm install npm@`*`version`* `–g`
- *`npm --version`*
- `sudo npm install -g @angular/cli@`*`versi`*
- *`ng version`*

More commands/settings for different versions may be needed, including compiling NodeJS yourself.



© AG SSE

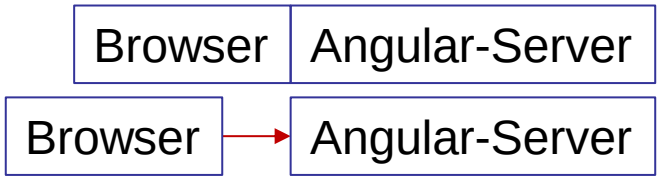
Contents

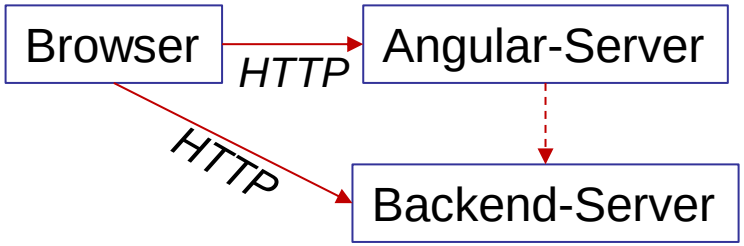
1. Introduction 2
2. Web Stacks 3
3. IDEs 9
4. Virtual Machines 15
5. A bit Node.js 21
6. Summary 28

© AG SSE

Security / CORS

- Usually: Same-Origin-Policy (SOP)
- Cross-Origin Resource Sharing (CORS)





Allows all, may be specific origin server name, IP address; may not work with localhost (Chrome)

- CORS-Plugin für Browser
- Backend server must declare CORS in HTTP-Header, e.g.,

```
Access-Control-Allow-Headers: X-Requested-With, Content-Type
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, DELETE, PUT, PATCH
```

A bit Node.js / express

- Works also with rather plain Node.js
  - Request handling via Endpoint-Routes and handlers
  - Various security packages, e.g., Helmet
- Express is a static web server extension for Node.js
  - Makes life a bit easier
  - Allows serving files such as HTML, CSS from a given directory
  - Supports various mechanisms for reading form headers
  - Supports data validation (we will skip this here)
  - …

© AG SSE

A Node.js example (1)

- Simple response to a get-Form without full webserver
- See Example `K2_01_NodeJs`
- Portable installation for Windows (more details in `index.html`)
  - Download and unpack portable Zip to *<N_DIR>*
  - Add *<N_DIR>* to `PATH` environment variable
  - Define `NODE_PATH` environment variable as *<N_DIR>*`\node_modules`
  - Execute `npm install express`
  - Start `node server.js` in example directory

    `We have started our server on port 5000`

  - Open `index.html` in browser and submit data.

We use here the express static server!

Rather simple result!

Hello World!

© AG SSE

A Node.js example (2)

- Index.html

Note port, path and method!

```
<!-- omitted HTML, head, body -->
<div class="form-container">
  <form id="registerForm"
   action="http://localhost:5000/api/helloworld" method="get">
  <!-- form as in K1_06_CSS -->
  </form>
</div>
```


© AG SSE

A Node.js example (3)

- server.js

```
// Load express module...
var express = require("express");

// Create express-appvar
app = express();

// Initialize Server with express-appvar
server = app.listen(5000, function () {
  console.log("We have started our server on port 5000");
});

// Configure a GET-Endpoint-Route
app.get("/api/helloworld", (req, res) => {
    console.log("`/api/helloworld` was called.")
    res.status(200) // Respond HTTP-200 to Client
    .send("<h1>Hello World!</h1>"); // Generate HTML-Output
})
```

Server network port!

Path and method!

Node.js handling requests parameters – body-parser

- Request-header parser for various formats except for `multipart/form-data`
- Import body-parser

```
var bodyParser = require('body-parser');
```

- Define parsers (here JSON + URLencoded)

```
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({
    extended: true
}));
```

- Access values via `req.body` etc.
- Install via NPM

https://github.com/expressjs/body-parser

© AG SSE

Node.js handling requests parameters - Multer

- Request-header parser for `multipart/form-data`
- Import multer

```
var multer  = require('multer');
```

- Define upload properties

```
var upload = multer({ dest: uploadDir });
```

- Apply multer

```
app.post("/submit", upload.single('File'), (req, res, next) => {
    // request data is in req.body
    // file is in req.file
}
```

- Install via NPM

https://github.com/expressjs/multer

© AG SSE

Task K2_01

Install your initial own VirtualBox-based development / server VM for an appropriate variant of the NEA*U-Stack, i.e.,

- An Ubuntu Linux from osboxes.org (see before, osboxes.org image on VMWare Player/Workstation)
- Node.js (Express.js may follow later)
- Angular

Test the installation of the individual components, their reachability via network etc. You may separate into server/development VM, later, …

© AG SSE

Contents

1. Introduction 2
2. Web Stacks 3
3. IDEs 9
4. Virtual Machines 15
5. A bit Node.js 21
6. Summary 28

© AG SSE

Summary

- Setup for Web application development
  - Various stacks
  - Initial experiments with Node.js
  - Recent IDEs
- Installation of VMs for Development and Deployment
- Test of node.js and database on the Deployment / Server VM

© AG SSE